

PySpark learning + revision roadmap [Part 1]

1. What is PySpark?

- **PySpark** is the **Python API for Apache Spark**.

It is used to **process and analyze very large amounts of data quickly** by distributing the work across multiple computers.

Simple example:

- Python → processes data mainly on one machine
- PySpark → processes huge data using multiple machines

👉 **PySpark = Python + Apache Spark** Commonly used in **Data Engineering, ETL, Databricks, Big Data, and data processing**.

2. Why it is required

PySpark is required when you need to **process huge amounts of data efficiently**.

Why PySpark?

- ⚡ **Fast** processing of large datasets
- 📊 Handles **millions/billions of records**
- 💻 Can use **multiple machines** together
- 🔗 Useful for **ETL/Data Pipelines**
- ☁ Widely used with **Databricks, AWS, Azure**
- 💼 Important skill for **Data Engineers**

Example: Processing 1 TB of customer data is difficult with normal Python/Pandas, but PySpark can distribute the work across machines.

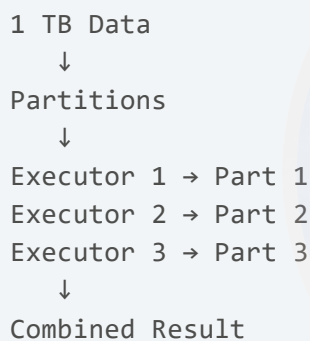
3. How PySpark works internally

Simple flow:

PySpark Code → Driver → Cluster → Executors → Tasks → Result

1. **Driver** – Runs your PySpark program and creates the execution plan.
2. **Cluster Manager** – Allocates resources/machines.
3. **Executors** – Run the actual data processing.
4. **Data is divided into partitions** – Each partition is processed separately.
5. **Tasks** – Executors process these partitions in parallel.
6. **Result** – Processed data is returned/stored.

Example:



```

1 TB Data
  ↓
Partitions
  ↓
Executor 1 → Part 1
Executor 2 → Part 2
Executor 3 → Part 3
  ↓
Combined Result
  
```

LinkedIn@AshishZope

👉 **Main idea: PySpark divides large data into smaller partitions and processes them in parallel.**

4. Basic PySpark Syntax

```

from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MyApp") \
    .getOrCreate()

df = spark.read.csv("data.csv", header=True, inferSchema=True)

df.show()
  
```

LinkedIn@AshishZope

Common syntax

```
df.select("name", "age")
df.filter(df.age > 25)
df.groupBy("city").count()
df.withColumn("new_col", ...)
df.orderBy("age")
```

LinkedIn@AshishZope

5. Multiple practical PySpark examples.

Create DataFrame

```
data = [
    (1, "Ashish", 50000),
    (2, "Rahul", 60000),
    (3, "Amit", 45000)
]

df = spark.createDataFrame(data, ["id", "name", "salary"])

df.show()
```

LinkedIn@AshishZope

Select columns

```
df.select("name", "salary").show()
```

LinkedIn@AshishZope

Filter data

```
df.filter(df.salary > 50000).show()
```

LinkedIn@AshishZope

Add new column

```
from pyspark.sql.functions import col

df = df.withColumn("bonus", col("salary") * 0.10)
df.show()
```

LinkedIn@AshishZope

Rename column

```
df = df.withColumnRenamed("salary", "monthly_salary")
```

LinkedIn@AshishZope

Sort data

```
df.orderBy(col("salary").desc()).show()
```

LinkedIn@AshishZope

Group By

```
df.groupBy("city").count().show()
```

LinkedIn@AshishZope

Aggregation

```
from pyspark.sql.functions import avg, max, min, sum

df.groupBy("city").agg(
    avg("salary").alias("avg_salary"),
    max("salary").alias("max_salary")
).show()
```

LinkedIn@AshishZope

Read CSV

```
df = spark.read.csv(
    "employees.csv",
    header=True,
    inferSchema=True
)
```

LinkedIn@AshishZope

Read JSON

```
df = spark.read.json("employees.json")
```

LinkedIn@AshishZope

Write Parquet

```
df.write.mode("overwrite").parquet("output/employees")
```

LinkedIn@AshishZope

Join two DataFrames

```
result = employees.join(
    departments,
    employees.dept_id == departments.dept_id,
    "inner"
)

result.show()
```

LinkedIn@AshishZope

Remove duplicates

```
df.dropDuplicates(["id"]).show()
```

LinkedIn@AshishZope

Handle NULL values

```
df.fillna({"salary": 0}).show()
```

LinkedIn@AshishZope

SQL with PySpark

```
df.createOrReplaceTempView("employees")

result = spark.sql("""
    SELECT name, salary
    FROM employees
    WHERE salary > 50000
""")

result.show()
```

LinkedIn@AshishZope

Real-world Data Engineering flow

```
CSV / API / Database
  ↓
PySpark Read
  ↓
Transform
  ↓
Filter / Join / GroupBy
  ↓
Data Quality
  ↓
Write Parquet
  ↓
ADLS / S3 / Data Lake
```

